

Moteur de backtesting event-driven en C++ moderne

Contexte

Ce projet a un objectif premier d'**apprentissage du C++**. N'ayant jamais écrit de C++, l'approche retenue est de découvrir le langage non pas via une série d'exercices isolés, mais à travers un livrable applicatif substantiel qui force à rencontrer, dans un ordre naturel, l'ensemble des mécanismes clés du langage : gestion mémoire, RAII, smart pointers, templates, STL, polymorphisme, move semantics, concurrence.

Le domaine retenu est la **finance quantitative** : conception et implémentation d'un moteur de backtesting pour stratégies de trading systématiques sur données historiques. Le choix du domaine est motivé par un contexte professionnel de quant Python / ML, ce qui permet de concentrer l'effort cognitif sur les spécificités du langage plutôt que sur la compréhension du problème métier. Le C++ apporte par ailleurs une réelle valeur ajoutée sur ce type de charge de calcul, ce qui rend le projet représentatif d'un usage industriel du langage et non purement pédagogique.

Objectifs

Objectifs pédagogiques

Couvrir les fondamentaux et les mécanismes distinctifs du C++ moderne dans un cadre applicatif cohérent : gestion mémoire et RAII, sémantique de valeur et de référence, ownership et smart pointers, polymorphisme (statique via templates et dynamique via héritage), move semantics, STL, concepts C++20, chaîne de compilation (CMake), tests unitaires, et outillage de profiling.

Objectifs techniques

Livrer un moteur de backtesting fonctionnel, modulaire, testé, et mesurablement performant. Le moteur doit permettre de définir une stratégie de trading, la simuler sur un historique de prix, et produire des métriques de performance standard (PnL, drawdown, Sharpe ratio). L'architecture doit être extensible à de nouvelles stratégies, de nouveaux instruments, et à des exécutions parallèles.

Périmètre fonctionnel

Le moteur simule l'exécution d'une stratégie sur un flux de données historiques en suivant une architecture **event-driven** inspirée des systèmes de trading réels : les données génèrent des événements de marché, la stratégie produit des signaux, un module de portfolio transforme les signaux en ordres, et un broker simulé exécute ces ordres et retourne des fills. Cette séparation permet à terme de basculer d'un backtest vers un environnement de paper trading ou de production avec un impact architectural minimal.

Le périmètre couvre les données OHLCV (barres) sur actions ou instruments à liquidité suffisante pour un modèle d'exécution simplifié. Les cas complexes (market impact, carnet d'ordres complet, données tick-by-tick) sont hors périmètre initial mais l'architecture doit rester compatible avec leur ajout ultérieur.

Architecture cible

Le moteur repose sur quatre composants principaux :

- **DataHandler** — lecture et itération sur les données historiques, émission d'événements de marché.
- **Strategy** — classe de base abstraite consommant les événements de marché et produisant des signaux ; les stratégies concrètes en dérivent.
- **Portfolio** — traduction des signaux en ordres, suivi des positions et du cash, calcul du PnL en temps réel.
- **Broker (ExecutionHandler)** — simulation d'exécution, application des coûts de transaction et du slippage, émission des fills.

Ces composants communiquent via un bus d'événements typés, ce qui découple leur cycle de vie et facilite le test isolé de chacun.

Roadmap

Phase 1 — Fondations et lecture de données

Mise en place de la chaîne d'outils (CMake, structure du repo, .clang-format, framework de test). Implémentation des types de base représentant une barre et une série temporelle, lecture de fichiers CSV, itération. *Objectif d'apprentissage* : syntaxe, STL de base (vector, string, chrono), organisation d'un projet C++.

Phase 2 — Stratégie et métriques de performance

Introduction d'une classe abstraite Strategy et d'une première implémentation concrète (SMA crossover). Calcul du PnL, du drawdown et du Sharpe sur la série de trades générée. *Objectif d'apprentissage* : héritage, polymorphisme dynamique, fonctions virtuelles, séparation interface / implémentation.

Phase 3 — Architecture event-driven

Refonte du moteur autour d'un bus d'événements (MarketEvent, SignalEvent, OrderEvent, FillEvent) et introduction du portfolio et du broker simulé. Gestion propre de l'ownership des événements et des ordres. *Objectif d'apprentissage* : smart pointers (unique_ptr, shared_ptr), std::variant, design patterns (observer, strategy), RAII.

Phase 4 — Généricité et instrumentation

Passage à une architecture générique via templates : le moteur devient paramétrable sur le type de barre, le type d'exécution, la précision numérique. Ajout de logs structurés et d'un profiling systématique. Benchmark comparatif face à un équivalent Python (pandas ou vectorbt). *Objectif d'apprentissage* : templates, concepts C++20, CRTP, std::chrono pour l'instrumentation, mesure de performance.

Phase 5 — Extensions

Selon les intérêts et le temps disponible, une ou plusieurs directions : parallélisation d'un grid search de paramètres via std::thread ou std::async ; implémentation d'un carnet d'ordres à limit-levels ; exposition du moteur à Python via pybind11 pour l'utiliser depuis un notebook ; ajout d'un modèle de coûts de transaction plus riche. *Objectif d'apprentissage* : concurrence, interopérabilité C++/Python, optimisation.

Stack technique

- **Langage** — C++17 minimum, C++20 privilégié pour concepts et ranges.
- **Build** — CMake.
- **Tests** — GoogleTest ou Catch2.
- **Formatage** — clang-format.
- **Compilateurs cibles** — GCC et Clang, avec warnings stricts (-Wall -Wextra -Wpedantic).
- **Dépendances externes** — minimales en Phase 1-2, potentiellement fmt pour le formatage et pybind11 en Phase 5.

Livrables

- Code source du moteur, organisé en bibliothèque réutilisable et exécutable de démonstration.
- Suite de tests unitaires couvrant les composants critiques.
- Documentation de l'architecture et de l'API.
- Un ou plusieurs notebooks / scripts d'exemple illustrant l'usage du moteur sur des stratégies simples.
- Rapport de benchmark comparant le moteur C++ à une implémentation Python de référence.